

AI PRD Template

AI Product Requirements Document

Module 5: PM for AI Projects | Week 7

ชื่อ / Name: ณัฐพลคัลภ์ กันทะสร Natpakan Kanthasorn

ชื่อ Product / Feature: ผู้ช่วยวิเคราะห์ Log และเฝ้าระวังระบบด้วย AI / Log Analysis AI

วันที่ / Date: Sat 30 May 2026

Exercise 1 | Broken Sprint

Case: The Churn Prediction Launch

FinoBank เพิ่งเปิดตัว Churn Prediction Feature ในแอปของตัวเอง ทีม PM วางแผนแบบ Agile sprint ปกติ ตั้ง Acceptance Criteria ไว้ว่า:

"Model ต้องมี Accuracy $\geq 85\%$ บน test set"

"Feature ต้องทำงานได้โดยไม่มี error ใน staging environment"

"Sprint ใช้เวลา 2 สัปดาห์ เหมือน sprint ทั่วไป"

ผ่านไป 3 เดือน ทีม Business Analyst รายงานว่า: ลูกค้า 40% ที่ model ทำนายว่าจะ churn ไม่ได้ churn จริง แต่ได้รับ retention offer ไปแล้ว ทำให้บริษัทเสี่ยงงบประมาณ retention ไปโดยเปล่าประโยชน์ นอกจากนี้ ลูกค้า 15% ที่ churn จริงๆ ไม่ได้รับ offer เลย

Model มี Accuracy 87% — เกิน threshold ที่ตั้งไว้ทุกอย่าง แต่ผลลัพธ์ทางธุรกิจแย่มาก

1. ระบุว่า PM ทำอะไรผิด — เขียน 2-3 จุดที่คิดว่าพลาด
2. เขียน Acceptance Criteria ใหม่ — แทนที่ criteria เดิมด้วย criteria ที่ดีกว่า ให้คิดถึง metric ที่ตรงกับเป้าหมายธุรกิจจริงๆ

แกะ Acceptance Criteria ใหม่ในแชทเมื่อเสร็จ

จุดที่ PM พลาด

"Model ต้องมี Accuracy $\geq 85\%$ บน test set"

- ไม่ได้บอกว่าเป็น accuracy สำหรับ case ไหน // recall / precision

"Feature ต้องทำงานได้โดยไม่มี error ใน staging environment"

- Ok, staging env ไม่ได้มีเคสจริงๆเข้ามา

"Sprint ใช้เวลา 2 สัปดาห์ เหมือน sprint ทั่วไป"

- Sprint อาจจะสั้นลง การสร้าง features หนึ่งใช้เวลาน้อยลง แต่ทดสอบบ่อยๆ

Acceptance Criteria ใหม่

1. Recall > 0.80 สำหรับ ลูกค้าที่จะยกเลิกจริงๆ จะต้องตรวจจับให้เจอ
 2. Precision > 0.70 สำหรับ ลูกค้าที่จะยกเลิก และควรได้รับ Offer
 3. ถ้ามี false positive (ถ้า detect ลูกค้าที่ไม่ได้จะยกเลิกจริงๆ) budget ในการให้ offer ต้องไม่เกิน 10% ของ offer ทั้งหมด
 4. Confidence score $< 60\%$ ให้มีนักลูกค้าสัมพันธ์มาตรวจสอบ
-

AI Product Requirements Document (PRD)

จาก AI Solution Canvas

01 Problem Statement | คำอธิบายปัญหา

อธิบายปัญหาที่ต้องการแก้ไข ใครได้รับผลกระทบ บ่อยแค่ไหน และส่งผลอย่างไรต่อธุรกิจหรือผู้ใช้
เริ่มจาก Canvas Section 1 แล้วเพิ่มความ specific ขึ้น: ระบุ segment ของผู้ใช้ให้ชัด และใส่ตัวเลขหรือข้อมูลถ้ามี

Problem Statement

ระบบ Tech ที่ประกอบด้วย Microservices หลายตัวที่ทำงานเชื่อมต่อกัน เมื่อเกิดเหตุการณ์ระบบล่ม หรือทำงานช้า ทีมวิศวกรต้องเสียเวลาหลายชั่วโมง เพื่ออ่าน log ซ้ำ service หลายตัวด้วยตัวเอง เพื่อหาต้นเหตุว่าเกิดจากโค้ดชุดไหน หรือเซิร์ฟเวอร์อะไร ปัญหาคือ log จำนวนไหลเข้าอย่างมหาศาล และเป็นความผิดปกติแบบ Multi-dimension (หลายๆปัจจัยร่วมกัน) และมักหลุดรอดการตรวจจับด้วย rule-based แบบเดิม ทำให้เวลาในการกู้ระบบยาวนานเกินไป

ใครได้รับผลกระทบ | Who is Affected

ระบุ user segment ให้ชัดเจน

- DevOps, Backend Engineer ที่ต้องคอยดูแลระบบ prod ของ platform
- ผู้ใช้ปลายทางของระบบที่ได้รับผลจาก downtime/latency และธุรกิจที่เสียหายหรือความเชื่อมั่นในช่วงระบบล่ม

บ่อยแค่ไหน / รุนแรงแค่ไหน | Frequency / Severity

ใส่ตัวเลขหรือความถี่ถ้ามี

- Frequency: ดึงตัวเลขจาก incident (จำนวน incident/month)
- Severity: แต่ละ incident ที่วิศวกรต้องเสียเวลาไล่ log -> ต้นทุนเวลาของวิศวกร + ต้นทุนตอน downtime ที่กระทบรายได้และผู้ใช้

02 Solution Overview + AI Approach | วิธีแก้ปัญหาและแนวทาง AI

อธิบายว่า solution ทำอะไร — AI ทำหน้าที่อะไรใน solution นี้ เป็น AI ประเภทไหน (Predictive / Generative / Classification / etc.)

ทำไมถึงเลือกใช้ AI แทน rule-based หรือ manual process? และ AI ทำงานร่วมกับผู้ใช้อย่างไร?

Solution Overview

ระบบจะรับ log ที่ไหลเข้ามาแบบ real-time จากทุก microservice นำมาประมวลผล และค้นหารูปแบบเพื่อตรวจจับจุดที่ผิดปกติ จากนั้นวิเคราะห์ต่อ ว่าความผิดปกติที่น่าจะมาจากโค้ดชุดไหน, service ไหน หรือเซิร์ฟเวอร์ใด จากนั้นหากเป็นความผิดปกติขั้นพื้นฐาน จะให้ AI แก้ไขความผิดปกตินั้นเลย หรือรอดูว่าเป็นปัญหาจริงๆ หรือไม่ เช่น ความผิดปกติของ network latency ที่เกิดขึ้นในช่วงเวลาที่ผู้ใช้มีความต้องการใช้มาก และหากเป็นความผิดปกติที่ร้ายแรง แล้วรวบรวมผลสรุปความผิดปกติส่ง Alert notification ที่สามารถระบุระดับความผิดปกติ, ความรุนแรง แล้วสรุปสาเหตุที่เป็นไปได้ในการแก้ไขให้ทีมวิศวกร เพื่อย่นเวลาในการหาต้นเหตุจากหลายชั่วโมงให้เหลือน้อยที่สุด

AI Type | ประเภท AI ที่ใช้

เช่น Classification, Predictive, Generative, Recommendation...

- **Anomaly Detection Predictive** -> ตรวจจับความผิดปกติจาก log system, error rate, API latency แบบ real-time
- **Classification (Supervised)** -> จัดระเบียบ root-cause category และ map ไปที่ service/commit ที่น่าสงสัย
- **Generative AI (LLM)** -> สรุปสาเหตุเป็นภาษาคน

Why AI? | ทำไมต้องใช้ AI

อธิบายว่าทำไม non-AI approach ไม่พอ

- Rule-based ดักได้เฉพาะ pattern ที่เรารู้และตั้งกฎล่วงหน้าไว้อยู่แล้ว แต่ความผิดปกติแบบ multi-dimension ตั้งกฎครอบคลุมไม่ได้
- Log volume สูง และเปลี่ยนรูปแบบตลอด ซึ่งการ maintain ด้วยมือไม่สเกล
- AI ต้องพิสูจน์ว่า incident ที่ rule-based แบบเดิมจับไม่ได้

User Flow | ผู้ใช้โต้ตอบกับ AI output อย่างไร

เขียนเป็น steps สั้นๆ

- Log (error, info, debug, API latency, error rate) ไหลเข้ามาในระบบแบบ real-time
- AI ตรวจพบ anomaly แล้วจัด root-cause ประเมิน severity และ confidence
- ระบบส่ง alert
- กรณีร้ายแรงและเคสแปลกๆ -> วิศวกรเปิด alert -> ตรวจสอบ -> ยอมรับหรือปฏิเสธ -> feedback กลับเข้าระบบเพื่อปรับปรุงโมเดล

03 Data Requirements | ข้อกำหนดด้านข้อมูล

ระบุข้อมูลที่ต้องการเพื่อให้ AI ทำงานได้ — ข้อมูลมาจากไหน ใครเป็น data owner ข้อมูลต้องใหม่แค่ไหน
คิดถึง data ที่ใช้ train model (historical) และ data ที่ใช้ run inference (real-time หรือ batch)

Training Data | ข้อมูลสำหรับ train model

What historical data is needed? Who owns it? How far back does it go?

Historical server logs -> Centralized logging platform -> Owner: Platform -> history >= 6-12 เดือน

Historical API latency & error-rate metrics -> Observability -> history >= 6-12 เดือน

Past incidents, root cause -> incident tracker -> ทุก incident ที่มีถูกวิเคราะห์

Git commit history -> version control -> เชิงค์กับ log

Inference Data | ข้อมูลที่ใช้ตอน run จริง

What data does the model need at runtime? Real-time or batch? Freshness requirement?

Real-time stream: server logs, API latency, error rate

Near-real-time: Latest commit

Freshness requirement: Log/Metric สดๆ ระดับ sec

Data Gaps and Dependencies | ข้อมูลที่ยังไม่มีหรือต้องขอจากทีมอื่น

ถ้าข้อมูลยังไม่มี หรือต้องขอจาก team อื่น ระบุ dependency ที่นี้

Log อาจมี Access token, password -> ต้องมี redaction, filtering layer ก่อนส่งเข้า model ไม่งั้นจะกลายเป็นช่องโหว่ความปลอดภัยทันที

Deadlock: จาก multi-thread อาจทำให้ server ค้าง โดยไม่มี error log -> เสริม health check/ heart beat signal เป็น input

04 Success Metrics | ตัวชี้วัดความสำเร็จ

แยก metrics ออกเป็น 2 ชั้น: Business Metrics (ผลลัพธ์ที่ business ต้องการ) และ Model Metrics (คุณภาพของ AI)

สำคัญ: ระบุ threshold ที่ยอมรับได้ด้วย — เช่น Recall ≥ 0.80 ไม่ใช่แค่ 'Recall สูงๆ'

Tip: กลับไปดู M3 — เลือก metric ให้ตรงกับ use case (Precision vs Recall tradeoff มีผลอย่างไรกับธุรกิจ?)

Business Metrics | ตัวชี้วัดทางธุรกิจ

e.g. Retention rate +5%, Cost per offer reduced 20%, Churn rate reduced within 90 days

Mean-time-to-detect -50%

Mean-time-to-recover -40%

Incident that rule-base can not detected but AI do > 0 (เพื่อพิสูจน์ว่า AI ดีกว่า rule-base)

Model Metrics | ตัวชี้วัดคุณภาพ model

e.g. Precision ≥ 0.75 , Recall ≥ 0.80 , F1 ≥ 0.77 — include threshold, not just metric name

Recall จับ incident จริงได้ ≥ 0.90 สำหรับ high-severity

Precision alert ที่ส่งเป็น incident จริง ≥ 0.70 overall กัน alert ขำซ้อน

MTTD (Mean-time-to-detect): Target $\leq$ min

Precision vs Recall Tradeoff

ใน product นี้ error แบบไหนแพงกว่ากัน? ถ้า model flag คนที่ไม่ได้ churn ว่า churn — เสียอะไร? ถ้า model พลาดคนที่ churn จริงๆ — เสียอะไร?

ถ้า model พลาด incident จริง (เป็น false negative) ระบบจะล่มโดยไม่มีใครรู้, downtime ยาว กระทบต่อผู้ใช้จริง เสียรายได้ ผิด SLA ขอดตกลง

ถ้า model flag ของปกติว่าผิดปกติ (เป็น false positive) วิศวกรเสียเวลาตรวจสอบ เกิด alert ซ้ำซ้อน ถ้าเยอะเกิน วิศวกรจะเริ่มเพิกเฉย กลายเป็นอันตรายทางอ้อม

05 Human-in-the-Loop (HITL) Requirements | ข้อกำหนด Human-in-the-Loop

ระบุว่า AI output ไหนที่ต้องให้คนตรวจสอบก่อนแสดงผล และ trigger conditions คืออะไร

เลือก HITL pattern ที่เหมาะสม: (1) Human reviews all outputs (2) Human reviews flagged outputs only (3) Human reviews on appeal

ออกแบบ escalation path: ถ้า output ถูก flag แล้วใครรับผิดชอบ และต้องตอบสนองภายในเวลาเท่าไร?

HITL Pattern ที่เลือก

วงกลม pattern ที่เหมาะกับ product นี้:

- ☐ Pattern 1 — Human reviews all outputs
- ☒ Pattern 2 — Human reviews flagged outputs only
- ☐ Pattern 3 — Human reviews on appeal

เหตุผลที่เลือก pattern นี้:

Alert ปริมาณมากเกินไปจะทำให้คนตรวจสอบทุกอัน แต่ incident ที่ความเสี่ยงสูง, confidence ต่ำต้องมีคนยืนยัน ก่อน ซึ่งตรวจเฉพาะที่ flag ตาม trigger ที่วัดได้

Trigger Conditions | เงื่อนไขที่ทำให้ต้องมีคนตรวจสอบ

e.g. Model confidence score < 0.65 | Transaction value > threshold | Customer LTV > X

Triggers must be objective and measurable — avoid 'when the model is not sure'

Model confidence < 0.65

AI signal ขัดแย้งกับ rule-based หรือ heartbeat ซีระบบค้างแต่ไม่มี error log (กรณี deadlock)

Who Reviews + SLA | ใครตรวจสอบและภายในเวลาเท่าไร

DevOps engineer ที่อยู่เวอร์

SLA: รับ Critical alert ตอบสนองภายใน 15 นาที

Escalation Path | ถ้า reviewer ไม่ว่าง ใครรับต่อ

Engineer manager

กรณีไม่มีใครคุมระบบเลย กำหนด auto-action ขั้นต่ำที่ปลอดภัย เช่น auto-rollback deploy ล่าสุด

06 Risks and Mitigations | ความเสี่ยงและการบรรเทา

แบ่ง risks ออกเป็น 3 ประเภท: Model Risk, Data Risk, Ethical / Business Risk

สำหรับแต่ละ risk ระบุ: ความเสี่ยงคืออะไร และจะบรรเทาหรือตรวจสอบอย่างไร

ใช้ AI Governance Checklist จาก Block 2 เป็น guide

Model Risk

What does the model get wrong? What is the worst-case failure scenario?

False Negative กรณีที่พลาด incident จริง และในกรณีที่แย่ที่สุดระบบล่มโดยไม่มี alert downtime ยาวๆ บรรเทาด้วยการตั้ง recall ≥ 0.90 (high-several, เพิ่ม heartbeat สำหรับกรณี deadlock ที่ไม่มี error log)

False Positive กรณี alert fatigue วิศวกรเลิกเชื่อ alert เฟิกเฉยของจริง บรรเทาด้วยการคุม precision ≥ 0.70 , ปรับ threshold จาก feedback

Wrong root cause -> ชี้ commit ผิด วิศวกรไล่ปัญหาผิดทาง เสียเวลามากกว่าเดิม บรรเทาด้วยการ flag confidence

Data Risk

What happens if training data is stale, incomplete, or biased toward a certain segment?

Access Token, password หลุดเข้า model บรรเทาด้วยการ redaction layer ก่อน และ scan log fields

Pattern เปลี่ยนหลัง deploy -> retraining schedule, model performance สม่ำเสมอ

Cold start: service ใหม่ยังไม่มีประวัติ -> rule-base ครอบคลุมช่วงต้น

Ethical / Business Risk

Could this AI disadvantage a customer segment? Could it be perceived as unfair?

Automation bias วิศวกรเชื่อ AI โดยยังไม่ได้ตรวจสอบ -> แสดง confidence + เหตุผลเสมอ

Accountability ตอนระบบล่มไม่มีคนคุม -> safe fallback action, audit trail ทุก auto action

07 Team and Roles | ทีมและบทบาท

ระบุว่าแต่ละ role รับผิดชอบอะไรใน project นี้ — ไม่จำเป็นต้องใส่ชื่อคน แต่ละ role และ responsibility
โฟกัสที่ decision points ที่สำคัญ: ใครเป็นคน sign off model performance? ใครเป็น HITL reviewer? ใครคือ escalation point?

Product Manager

สร้าง PRD นิยาม success metric + threshold, ทำ scope, ให้สัญญาณว่า model performance + business metric ผ่านก่อนจะ launch

ML Engineer / Data Scientist

ออกแบบ/เทรน anomaly detection + classification + LLM summarizer

รายงาน Precision/Recall

เสนอผลให้ PM ตรวจสอบ

Data Engineer

สร้าง log/metric pipeline -> streaming

Redaction, filtering layer

จัดการความ freshness/availability ของ interface data

เชื่อม git & heartbeat signals

Designer

ออกแบบ alert UX ให้สื่อถึง severity + confidence + เหตุผลที่ชัดเจน

ทำให้ flow acknowledge/ false alarm feedback

Business Stakeholder

สร้าง SLA และ downtime impact, อนุมัติ trade-off precision/recall
เป็น escalation point สูงสุด
อนุมัติ fallback auto-action

HITL Reviewer

DevOps
On-call SRE

PRD Status Check

- ☒ Draft complete — all 7 sections have content
- ☐ Needs data section — Section 03 still incomplete
- ☐ Needs metrics — Section 04 thresholds not defined
- ☐ Ready for prototype — PRD is solid enough to build from